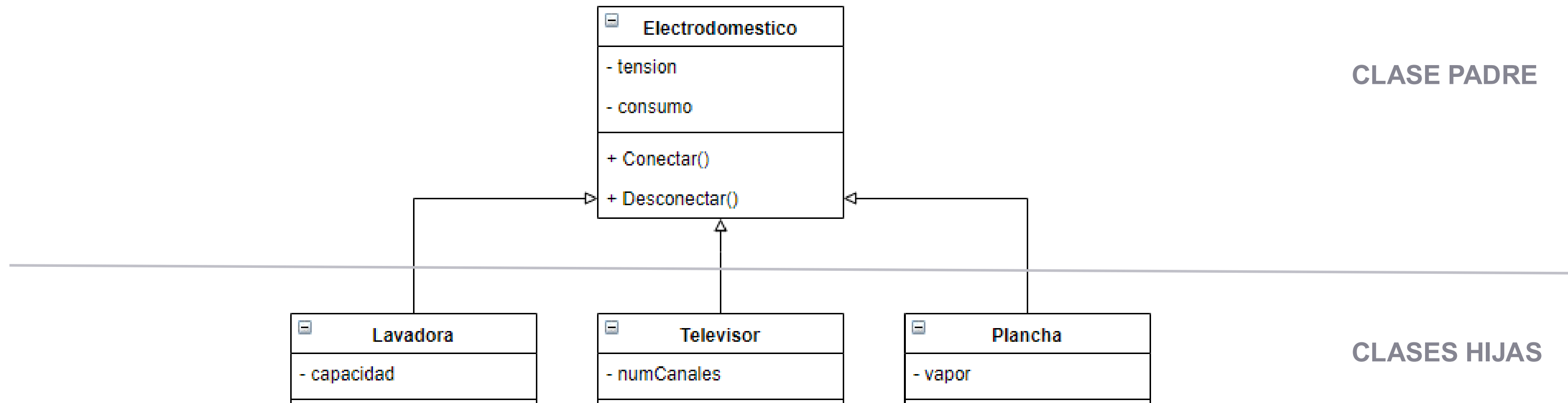


Diagrama de clases (Segunda parte)

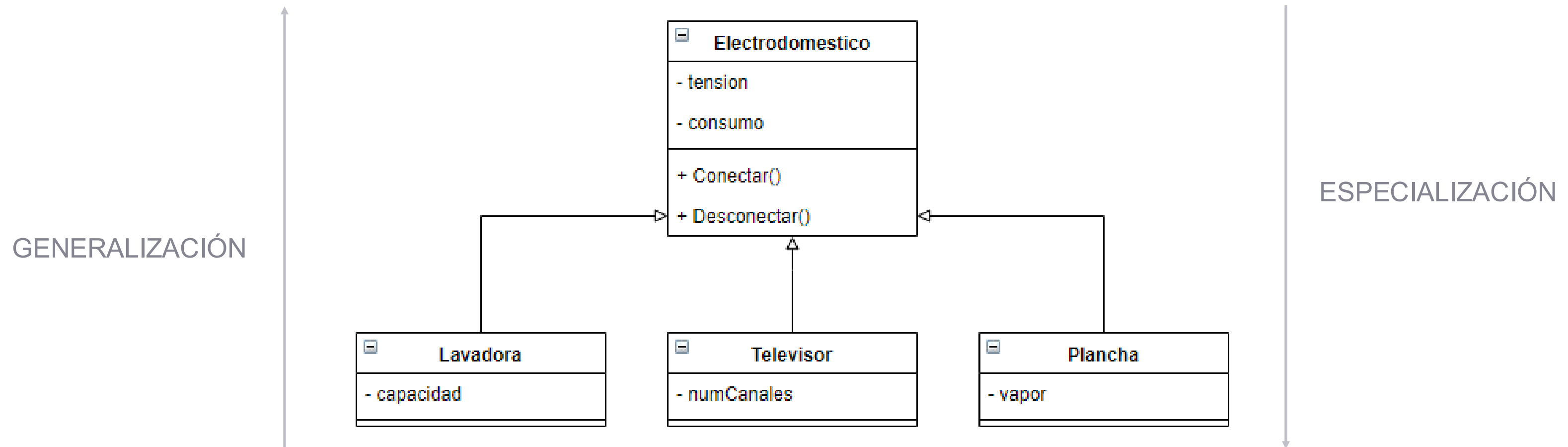
Herencia

La relación de herencia se representa mediante un triángulo en el extremo de la relación que corresponde a la clase más general o clase “padre”. En el caso de clases abstractas, el nombre de la clase padre se representa en tipografía itálica.



Herencia

Cuando la herencia se mira desde el hijo hacia el padre se denomina Generalización. Ahora si se mira desde el padre hacia el hijo, se denomina Especialización.

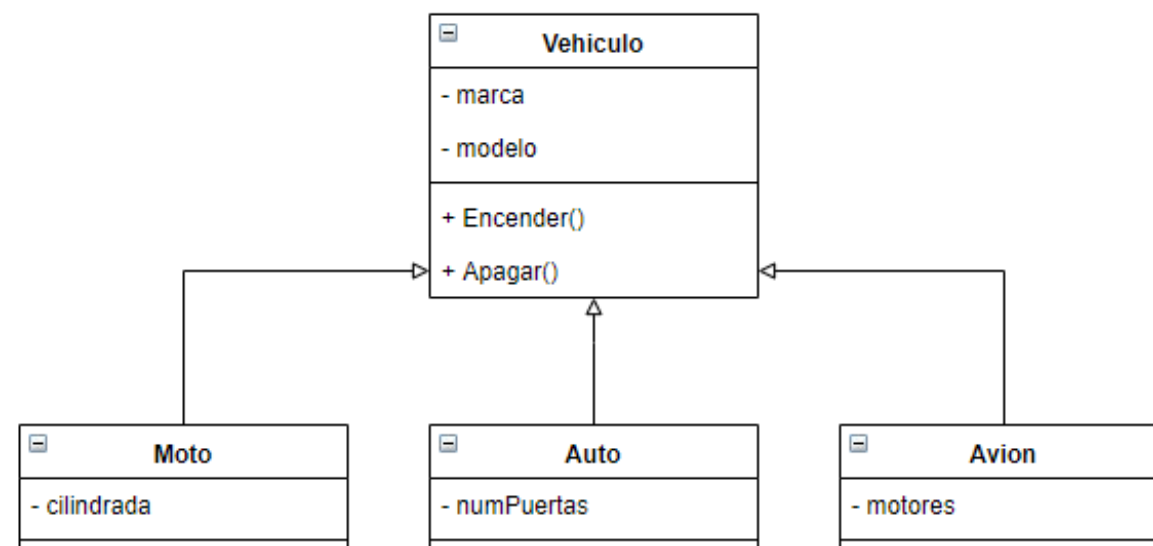


Herencia

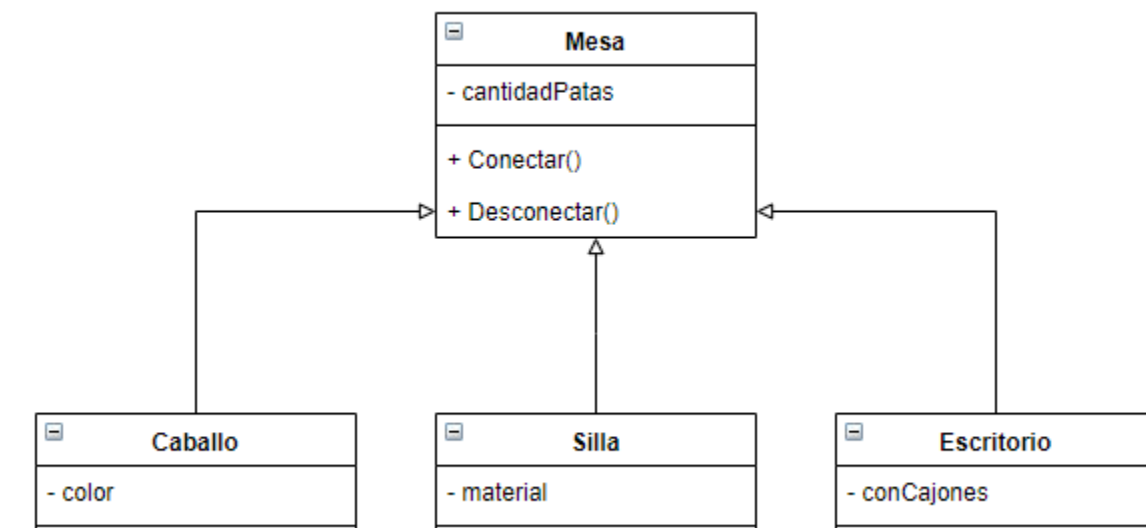
Según Barbara Liskov, en el artículo *Data Abstraction and Hierarchy*, existen dos tipos de herencia: herencia de tipo y herencia de uso.

La **herencia de tipo** responde a la pregunta: “¿es un?”, mientras que la herencia de uso no responde a ese interrogante..

La **herencia de uso** es muy utilizada por los programadores para ganar tiempo y poder reutilizar un método o atributo ya desarrollado.



TIPO



USO

Herencia

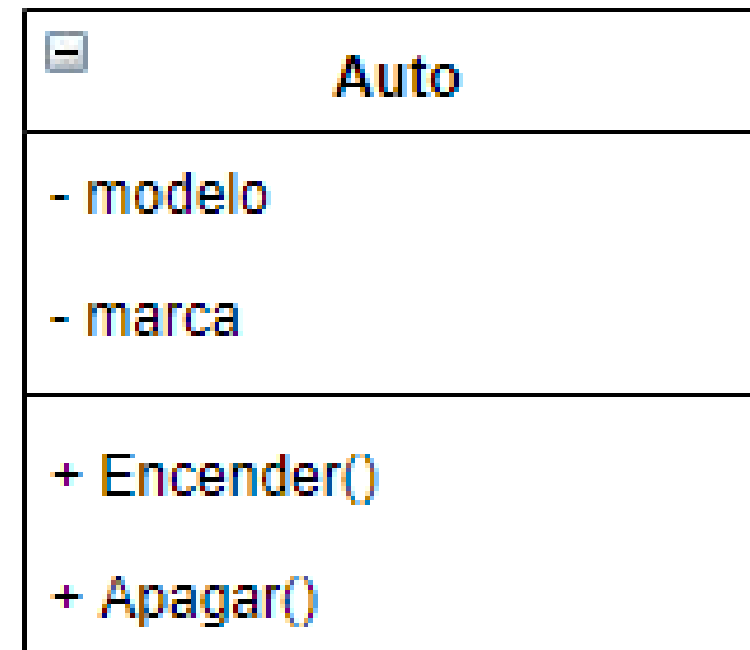
Según Alan Snyder, en el artículo *Encapsulation and Inheritance in Object-Oriented Programming Languages*, la herencia rompe el encapsulamiento.

Al heredar de su padre, una clase accede al contenido de sus atributos y a la implementación de sus métodos.

Propiedades estáticas o dinámicas

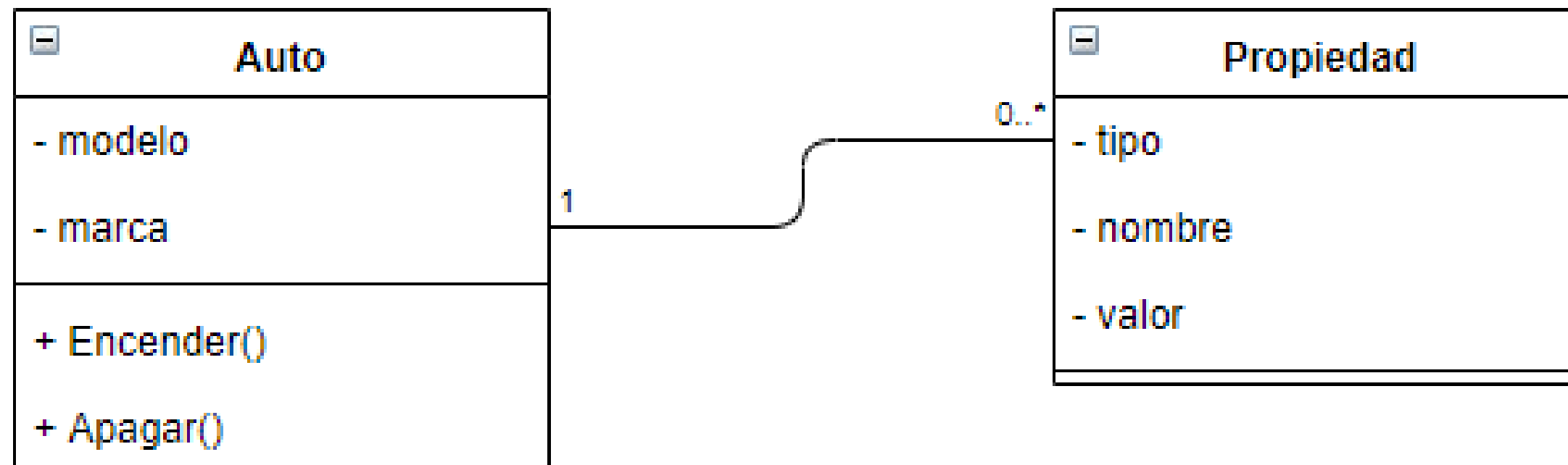
Según el artículo *Dealing with properties* existen muchas maneras de manejar propiedades o tributos en las clases.

Las propiedades estáticas son las que se abordaron en módulos anteriores, creándose en tiempo de diseño.



Propiedades estáticas o dinámicas

También es posible crear propiedades en tiempo de ejecución, las cuales son propiedades dinámicas. Existen muchas formas, pero la más utilizada es la siguiente:



Roles

Según Martin Fowler en el artículo *Dealing with roles*, existen 5 formas de manejar roles, en los cuales se presenta ventajas y desventajas.

Todas son posibilidades a la hora de manejar roles, lo importante es ver cuál es la forma apropiada para manejar una determinada situación.

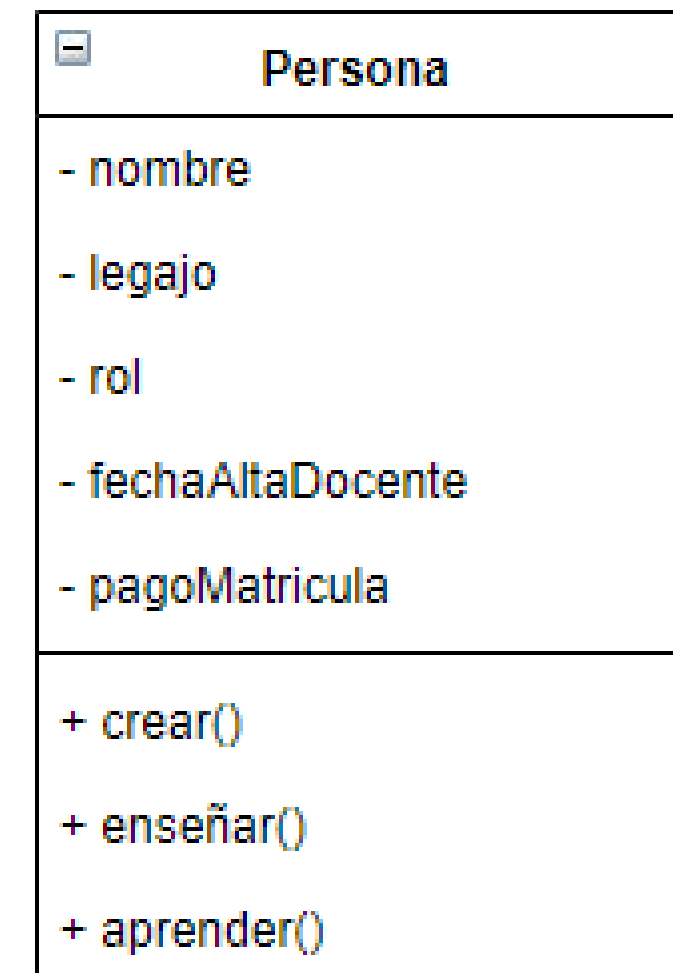
Roles

Single Role

En una sola clase metemos todos los roles (por ejemplo, Alumno y Docente), ya sea respecto a los atributos como a los métodos. Siempre se precisa un atributo para indicar qué tipo de rol va a tener la clase.

La **ventaja** hace referencia a la simplicidad, ya que en una sola clase se posee todo lo necesario.

La **desventaja** es difícil de mantener. Cada método posee dentro un if, que controla cómo se va a ejecutar ese método para ese rol. Si se agregara otro rol, sería necesario modificar todos los métodos para agregarle un if a cada método.



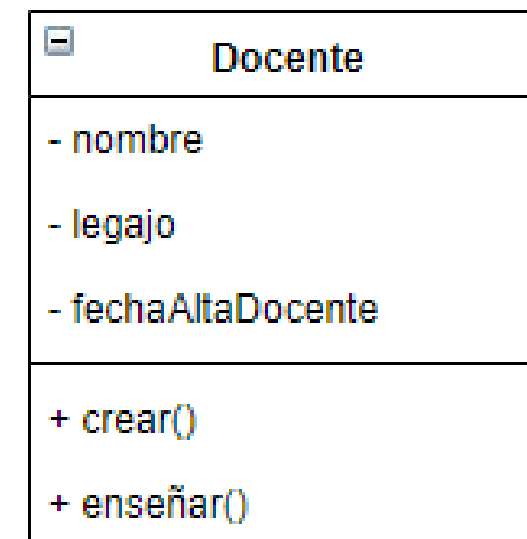
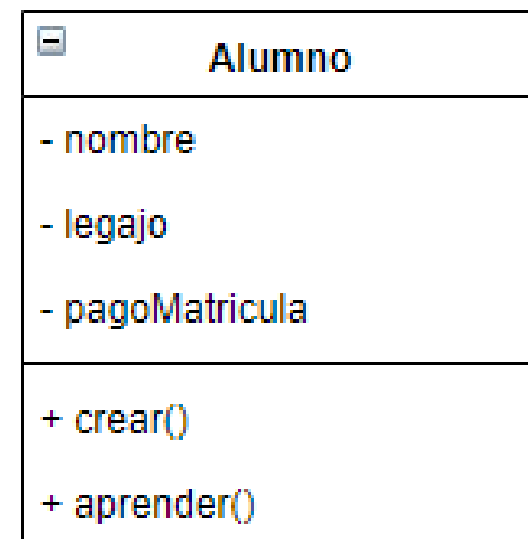
Roles

Separated Role

Se maneja una clase por cada rol (por ejemplo, Alumno y Docente).

Ventaja: es muy simple, muy intuitivo y se suele usar sin saber que ya se encuentra previamente diseñado.

Desventaja: se poseen atributos (nombre y legajo) y métodos duplicados (crear).

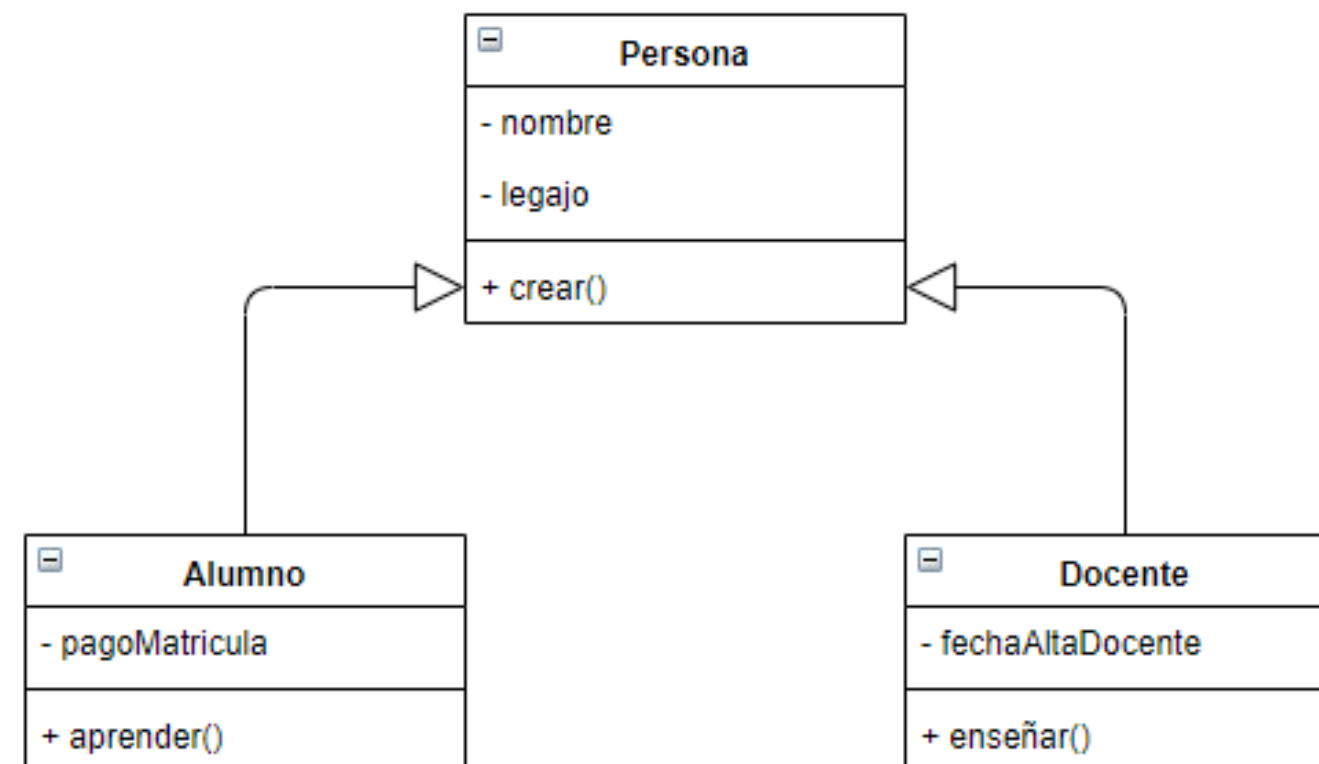


Roles

Role Subtype

Se agrupa en una clase padre los atributos y métodos en común. **Ventaja:** no se poseen atributos y métodos duplicados.

Desventaja: la persona no puede tener más de un rol al mismo tiempo.



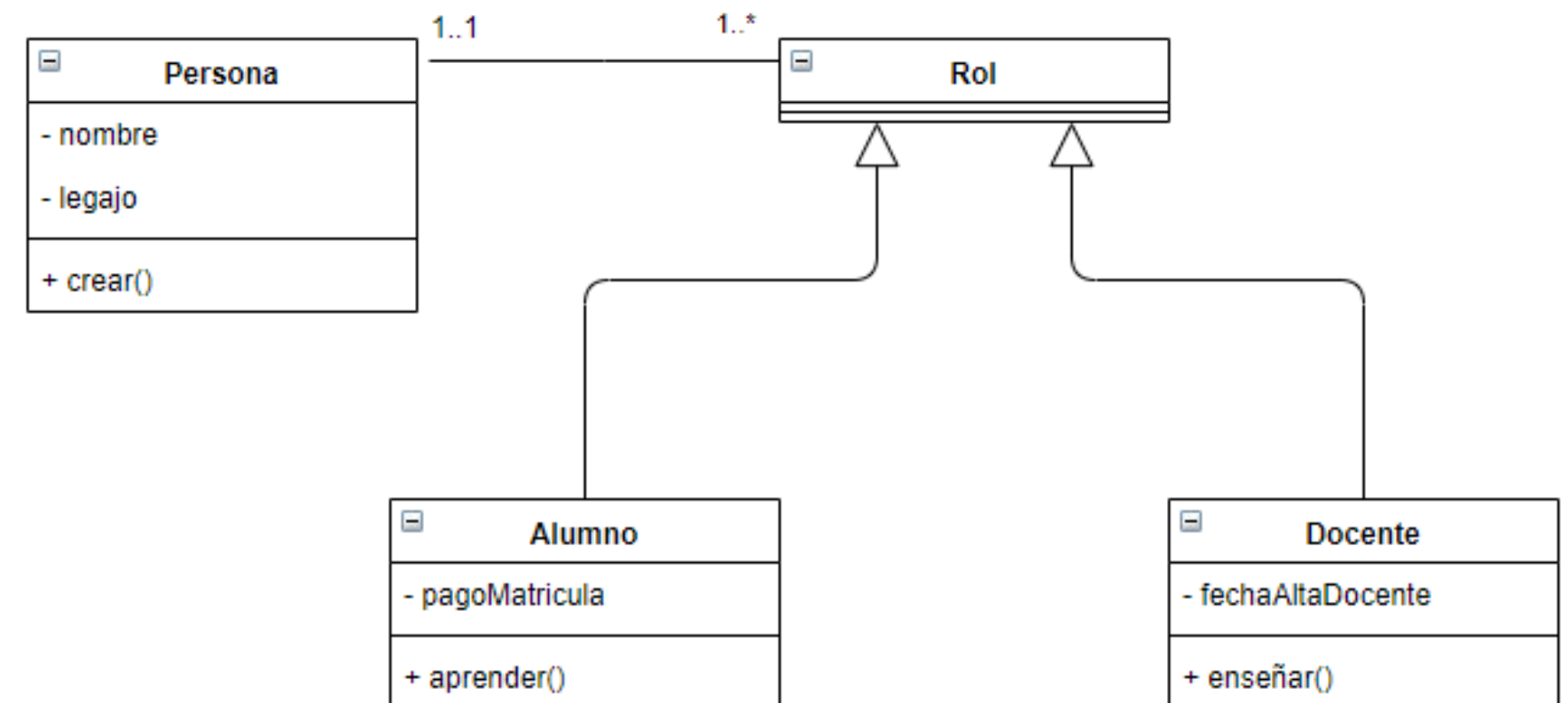
Roles

Role Object

Se genera un type object para poder manejar varios roles para una sola persona.

Ventaja: se pueden manejar varios roles para una sola persona.

Desventaja: se pueden manejar varios roles, pero no pueden definirse varios contextos.



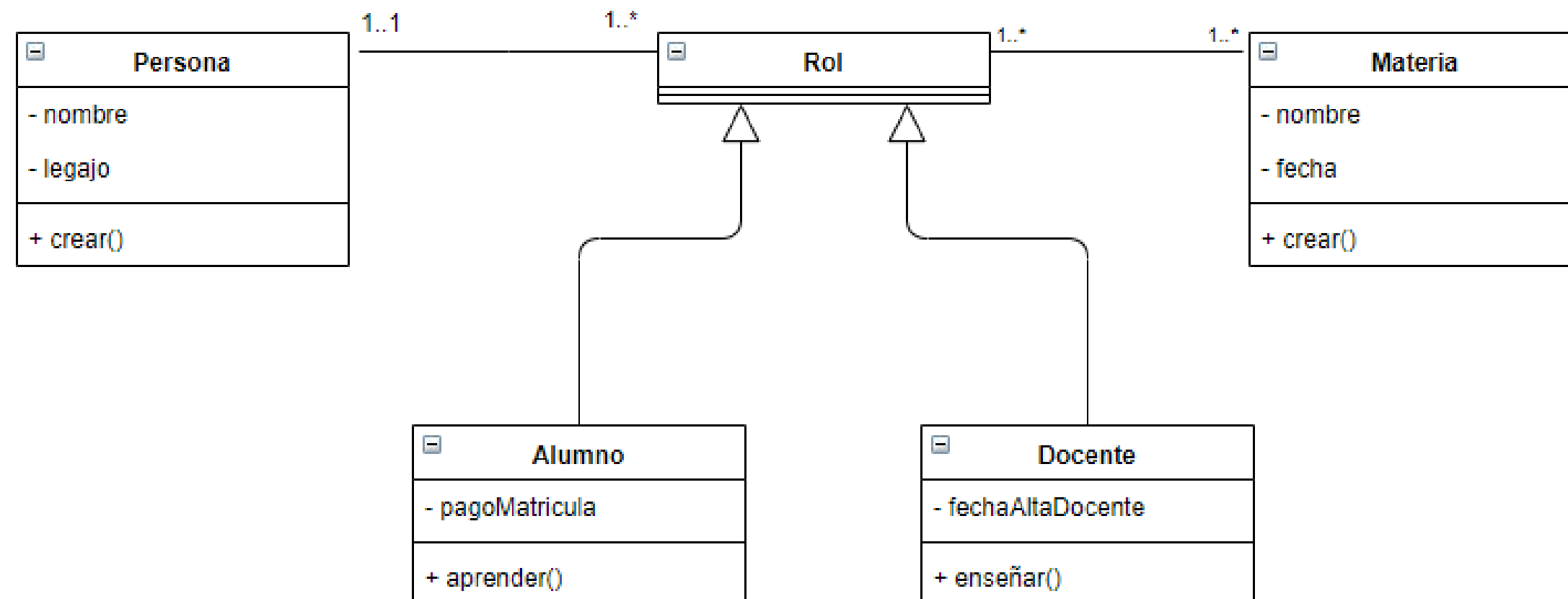
Roles

Role Relationship

Se agrega un contexto.

Ventaja: se maneja el contexto.

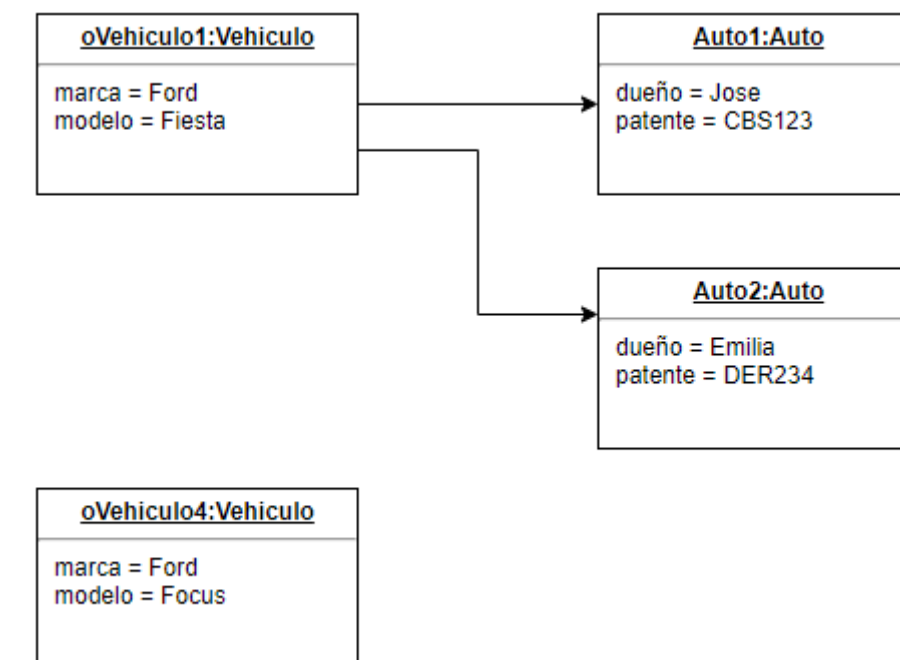
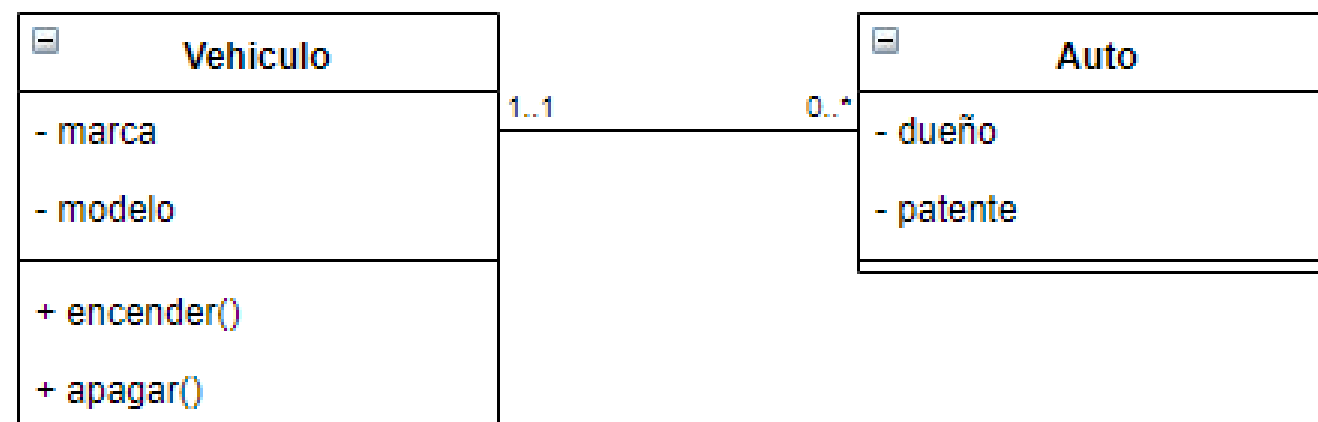
Desventaja: implica más complejidad.



Type Object

Según Ralph Johnson y Bobby Woolf en el artículo *The Type Object Pattern*, el Type Object desacopla instancias de sus clases para que estas puedan ser implementadas como una instancia de una clase. Type Object permite crear nuevas clases dinámicamente en tiempo de ejecución porque estas son instancias y también permite al sistema crear instancias de esas class-like instancias.

A veces una clase no solo requiere un número desconocido de instancias, sino que también una cantidad desconocida de subclases. Un objeto puede crear nuevas instancias, pero usualmente no puede crear nuevas clases sin tener que recompilar.





© Universidad de Palermo

Prohibida la reproducción total o parcial de imágenes y textos.